

FedChain: Blockchain-assisted Federated Learning Framework for Secure Resource Allocation in Device-to-Device Communication

Ziyankhan Pathan*, Mohammad S. Obaidat^{††}, *Life Fellow of IEEE*, Trupesh Patel[†], Sakshi Chavda[‡], Jigna Hathaliya[§], Anuja Nair[¶], Rajesh Gupta^{||}, Sudeep Tanwar^{**}

^{*‡§¶}Department of Computer Science and Engineering, Institute of Technology, Nirma University, Ahmedabad, India

^{††}Distinguished Professor at the King Abdullah II School of Information Tech., The University of Jordan, Amman 11942,

School of Computer and Comm. Engineering, University of Science and Technology Beijing, Beijing 100083, China,

Dept. of Computational Intelligence, School of Computing, SRM University, Kattankulathur 603203, TN, India

and School of Engineering, The Amity University, Noida, UP 201301, India.

[†]Department of Computer Engineering, Karnavati University, Ahmedabad, Gujarat, India

^{||**}Department of Computer Engineering, Marwadi University, Rajkot, Gujarat, India

Emails: *24bce146@nirmauni.ac.in, ^{††}msobaidat@gmail.com, [†]24ptphde285@nirmauni.ac.in, [‡]24bce169@nirmauni.ac.in,

[§]jigna.hathaliya@nirmauni.ac.in, [¶]anuja.nair@nirmauni.ac.in, ^{||}rgupta.ap@gmail.com, ^{**}sudeeptanwar149@gmail.com

Abstract—In this paper, we propose FedChain, a secure and decentralized blockchain-enabled federated learning (FL) framework for resource allocation (RA) in device-to-device (D2D) communication networks. Traditional D2D RA methods are based on centralized processing, which introduces privacy risks, single-point failures, and limited scalability. FL decentralizes model training, but remains vulnerable to model tampering and insecure parameter exchange between clients and the server. To overcome these challenges, FedChain integrates a permissioned Hyperledger Fabric (HF) blockchain to provide traceable, auditable, and tamper-proof model update transactions. Additionally, the interplanetary file system (IPFS) is employed to store model parameters off-chain, while only their cryptographic content identifiers (CIDs) are recorded on-chain to minimize blockchain storage overhead. The framework applies federated averaging (FedAvg) for collaborative model aggregation without exposing raw data. An experimental evaluation compared the results of 5 models, where FedChain achieves >95% global accuracy using Random Forest (RF) and eXtreme Gradient Boosting (XGBoost) algorithms. It helps improve security, tamper-resistance, model consistency, and communication efficiency. These results indicate that FedChain offers a scalable and secure foundation for decentralized RA in future 6G-enabled D2D networks.

Index Terms—Resource allocation, federated learning, IPFS, D2D communication, model aggregation, Hyperledger Fabric

I. INTRODUCTION

Device-to-device (D2D) communication has become an essential component in next-generation wireless networks due to its ability to enhance spectrum utilization, reduce end-to-end latency, and support high data-rate applications at the network edge. Effective resource allocation (RA) in D2D systems is

crucial for maintaining link reliability and interference control. However, many existing RA strategies depend on centralized controllers, making them susceptible to privacy leakage, scalability limitations, and single-point failures. This central dependency becomes increasingly problematic as the number of edge devices grows, leading to increased computation and communication burden on the central entity.

To overcome the drawbacks of centralized decision-making, federated learning (FL) has emerged as a promising solution to decentralize intelligence in D2D networks by allowing devices to collaboratively train a shared model without revealing their local data. While FL preserves data privacy, it still relies on a parameter aggregation server that becomes a potential target for model poisoning, tampering, and manipulation attacks. Moreover, the absence of transparent verification mechanisms across participating clients makes trust management challenging. To address these shortcomings, blockchain technology has gained attention as a distributed ledger that provides immutability, traceability, and decentralized consensus, making it a strong candidate to enhance the reliability and trustworthiness of FL-based systems.

Many researchers are working to perform D2D RA, such as Shamaei *et al.* [1] used matching theory to address co- and cross-tier interference in D2D-enabled cellular networks. They assumed a simple channel and didn't consider irrational behaviour. To address this, researchers moved to machine learning (ML) and deep reinforcement learning (DRL) methods such as deep q-networks (DQN)-based optimization [2], and ML-based 5G network slicing [3] that improved adaptability

and performance, but kept a centralized structure, causing exposure to single-point failures and privacy risks. Hybrid frameworks suggested by Noman *et al.* [4], combined FL with optimization algorithms and DRL to achieve energy-efficient and decentralized RA. Similarly, Guo *et al.* [5] combined FL with DQN for distributed RA policies trained locally on user devices. However, it led to vulnerabilities like model poisoning and parameter tampering due to insecure communication channels. To overcome these issues, blockchain can be integrated to increase the security of FL systems. Zhang *et al.* [6] used blockchain to secure model aggregation and participant selection. Authors in [7] proposed blockchain-based DRFL (BDRFL) framework with multi-subchain consensus mechanisms for reliability.

However, existing models remain reactive, optimizing resources only after observing network states, and use generic blockchain infrastructures not tailored for telecom applications. Ensuring trust, integrity, and transparency in FL-based D2D resource allocation remains an open challenge. To overcome this, our proposed framework introduces a predictive and proactive architecture using FL to predict optimal D2D RAs before commitment. By integrating Hyperledger Fabric (HF), a permissioned blockchain ensuring traceability and tamper resistance, it provides a secure and scalable solution for D2D resource management. The key contributions in this paper are:

- We proposed the FedChain framework, a secure, decentralized blockchain-integrated FL framework that eliminates central trust and single-point failure in D2D RA.
- We employ IPFS for off-chain model parameter storage, recording only CIDs on HF to ensure scalability, integrity, and low communication overhead.
- We integrated FedAvg for collaborative training, achieving >95% global accuracy from XGBoost, and RF models while improving robustness, privacy protection, and convergence stability.

The rest of the paper is organized as follows: Section II presents FedChain framework. Section III presents the experimental results. Section IV concludes the paper and outlines the scope for future work.

II. PROPOSED FRAMEWORK

In this section, we discuss the proposed framework, which contains a 3-layer architecture: data acquisition and Local Training Layer, Blockchain Layer, and Model aggregate Layer as visualized in Fig. 1. We discuss the algorithm used for calculating the resource optimization score (ROS) predicted by ML models.

A. Data Acquisition And Local Training Layer

This layer consists of an interface with end-to-end user and managing intelligence at the local client tower.

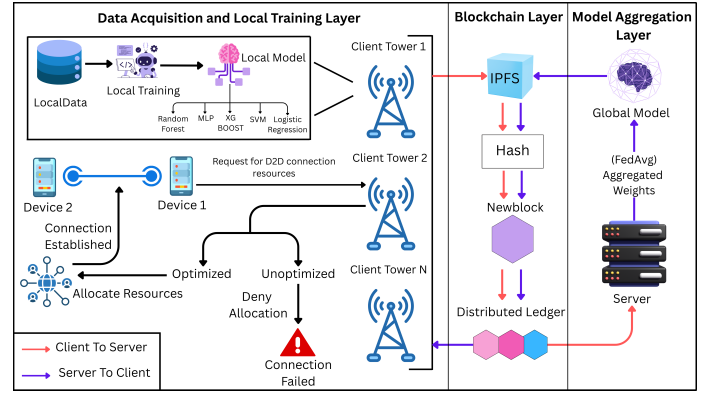


Fig. 1: Proposed framework

1) *D2D Request Handling*: When a device (Device 1) sends a request for sharing D2D communication resources with another device (Device 2), it is received by the local client tower. The tower uses its ML models to analyze the request parameters and decides whether the proposed allocation is "Optimized" or "Unoptimized" by reaching out consensus on the ROS calculated using all 5 models. If the model predicts "Optimized," the tower allocates the resources, and the D2D connection is successfully established. If the model predicts "Unoptimized," the tower denies the allocation, resulting in a "Connection Failed" state for that request.

2) *Local Model Training*: At the start, we had a dataset with 1000 rows of Social-Aware-Resource-Allocation data. As part of the FL mechanism, each client represents a local network tower that has 5 ML models implemented on local data available. For FL, we conducted 25 rounds of local training and model aggregation. In each round we randomly split the data, keeping 70% for training and 30% for testing for each client. In each round, we begin by distributing the current global model state to each client. The 5 models implemented are: RF, Multilayer Perceptron (MLP), XG Boost, Support Vector Machine (SVM), and Logistic Regression (LR). For pre-processing, the data was normalized using Min-Max Scaling (MMS) [8] for ROS composition and Standard Scaling (SS) [8] for training.

During training, we try to find parameters for our objective function that calculate ROS and minimize overfitting. We used loss functions and a regularization penalty for this. For instance, LR and MLP models are trained by minimizing the Binary Cross-Entropy (BCE) Loss [9]. Eq. 1 calculates BCE Loss:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (1)$$

Where L_{BCE} is BCE Loss, N is total number of samples, y_i is true label and $\hat{y}_i$ is predicted label.

Similarly, SVM minimizes the Hinge Loss to find a maximum-margin hyperplane.

To prevent complexity, LR, MLP, and SVM incorporate L_2 Regularization. On the other hand, RF doesn't optimize loss. Instead, it builds an ensemble of trees by recursively splitting features to reduce node impurity, measured by the Gini Impurity Score (GIS).

Out of all 5 models, the weights and bias of only LR and MLP can be saved, so the parameters of those models only go through the blockchain layer to the server. Remaining models like RF, SVM, and XG Boost go to the server without going through the blockchain layer.

B. Blockchain Layer

This layer obtains the data from the data acquisition layer to ensure the confidentiality and integrity of the model parameters. The role of a client is to upload the model parameters (MP) to the blockchain. The server aims to extract the MP from the blockchain and aggregate the MP from the global model (GM). In the above layer number of clients $C_N \in \{1, 2, \dots, N\}$ train their local model (LM_i) with MP using weights (W_i) and biases (B_i) $MP \in \{W_i, B_i\}$. Here, each client (C_e) trains their LM using MP.

$$C_e \xrightarrow{\text{trains}} LM \subseteq MP \quad (2)$$

$$MP \in \sum_{i=1}^N WB \quad (3)$$

In Eq. 3, the client uploads the WB of MP in the HFB. Moreover, the client also collects the GM's MP to synchronize their local model. Here, 100 clients update their WB at every update of the global model (GM). Here, clients update their WB in 25 rounds, resulting in a large amount of data. This large data consumes a significant memory on the blockchain, lowering the performance of the proposed framework.

With this concern, the proposed framework uses a decentralized file system such as IPFS, in which each client stores their updated WB in all 25 rounds.

$$\forall C_n \sum_{i=1}^N WB \xrightarrow{\text{stores}} IPFS \quad (4)$$

Eq. 4 displays the storage process of WB of all clients C_n in IPFS. Later, IPFS generates a content identifier (CID) for the uploaded data.

$$CID = H(C_{ce} \parallel \sum_{i=1}^N WB) \quad (5)$$

Eq. 5 outlines a CID computation process. Later, only this CID is stored in the HFB. Further, this CID is stored in the HFB. In this way, we stored only a hash of the WB on the HFB instead of storing a large amount of data. Hence, the proposed

framework only requires minimal storage in the blockchain to store this data. Consequently, this minimum storage helps to improve the performance of the proposed framework.

To access the data from the HFB, the server sends a request to the HFB. Here, the membership service provider (MSP) verifies the server's credentials and accepts the request.

$$MSP \rightarrow V_r(S_{ce}, SID) = \begin{cases} 1, & \text{if } (S_{ce} \wedge SID) \text{ are valid} \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

Eq. 6 shows the verification process (V_r) of the server by their certificate S_{ce} , and serverID SID . Further, the server accesses the CID of each client from the HFB. Later, the server uses this CID to access the WB from the IPFS. Moreover, the server aggregates the WB of all C_N , and creates a GM.

$$GM = \frac{1}{N} \sum_{i=1}^N WB \quad (7)$$

Eq. 7 illustrates the aggregation process of all C_N to aggregate WB in a GM. Following, these GM's WB_i values are stored in the IPFS, where IPFS generates a CID. Later, this CID is stored in the HFB, where a specific client extracts the CID of the updated WB from HFB. Further, C_e sends a request to HFB to access the GM's WB . MSP verifies and accepts the client's request based on their credentials.

$$MSP \rightarrow V_r(C_{ce}, C_eID) = \begin{cases} 1, & \text{if } (C_{ce} \wedge C_eID) \text{ are valid} \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

Eq. 8 presents the client verification process. MSP verifies the clientID (C_nID) based on their certificate (C_{ce}). Next, each client uses these updated WB to train and synchronize their model. Therefore, server and local clients will receive secure and the latest WB_i from the HFB. Blockchain ensures that server and local clients will only get the original weights and biases. In this way, the server and clients will train their global and local models while ensuring their security and privacy.

1) *Secure data storage of model parameters to Blockchain:* Algorithm 1 illustrates the client and server interaction with the HFB to access the MP. Initially, users login to HFB with their certificate (U_{ce}) for the interaction. *UploadModelP()* procedure is used to upload model parameters into HFB. First, we match userIDs with a specific C_eID or SID . Based on successful verification of the user, userIDs can upload their MP, such as WB . Suppose the userID matches C_eID , C_eID stores the WB to IPFS. IPFS returns CIDs of those WB for a specific C_e . Further, C_{ce} , CID_{WB} and timestamp (Ts) saved in data variable. Later, this data is stored in the HFB using `ctx.stub.putState()` method by converting string data into JSON. Suppose userID matches with SID , then SID uploads the WB to the IPFS. IPFS returns CID of WB . Next, S_{ce} , CID of

Algorithm 1 Client and server interaction with the HFB

Input: Upload model parameters (WB) of server and local clients to the Hyperledger Fabric Blockchain (HFB)

Output: Access model parameters WB from IPFS and HFB .

```
1: procedure UPLOADMODEL(ctx, clientID, WB, serverID)
2:   while (True) do
3:     if (UserID == clientID) then
4:       clientID  $\rightarrow$  stores  $WB$  to IPFS
5:       IPFS  $\rightarrow$  returns  $CID_{WB}$ 
6:       data  $\rightarrow C_{ce}, CID_{WB}, Ts$ 
7:       ctx.stub.putState( $C_{ce}$ ), JSON.stringify(data)  $\rightarrow HFB$ 
8:     else
9:       if (UserID == serverID) then
10:        serverID  $\rightarrow$  stores  $GW_{B}$  to IPFS
11:        IPFS  $\rightarrow$  returns  $CID_{GW_{B}}$ 
12:        GM  $\rightarrow S_{ce}, CID_{GW_{B}}, Ts$ 
13:        ctx.stub.putState( $S_{ce}$ ), JSON.stringify(GM)  $\rightarrow HFB$ 
14:      end if
15:    end if
16:  end while
17: end procedure
18: procedure GETMODELPARAMS(ctx, clientID, serverID)
19:   while (True) do
20:     if (UserID == clientID) then
21:       ctx.stub.getState(GM)  $\rightarrow HFB$ 
22:       clientID  $\rightarrow$  receives  $CID_{GW_{B}}$  from GM
23:       ClientID  $\xrightarrow{\text{submitsCID}}$  IPFS
24:       IPFS returns  $GW_{B} \rightarrow$  ClientID
25:     else
26:       if (UserID == serverID) then
27:        ctx.stub.getStateByClientID( $WB$ )  $\rightarrow HFB$ 
28:        serverID  $\rightarrow$  receives  $CID_{WB}$  from all clientID
29:        serverID  $\xrightarrow{\text{submitsCID}}$  IPFS
30:        IPFS returns  $WB \rightarrow$  for each clientID
31:      end if
32:    end if
33:  end while
34: end procedure
```

GW_{B} and Ts saved in GM variable. Following, this GM is stored in HFB using ctx.stub.putState() method.

GetModelParams() procedure is used to fetch MP, including WB values from HFB . Suppose user ID matches C_eID , then they receive GM using ctx.stub.getState() method from HFB . Later, we extract CIDs from GM. Further, the C_eID submits CID to IPFS to get GW_{B} . Further, a specific client can collect the GW_{B} , and update their WB value to train their LM. In this way, the C_e can train and synchronize their LMs. Suppose, user ID matches the SID, then SID receives the WB of the specific client by using ctx.stub.getStateByClientID() from HFB . Following, SID extracts CID_{WB} from all C_nID , and receives WB for each client. In this way, the server aggregates its GM by collecting all WB from each c_e .

C. Model Aggregation Layer

This layer consists of global model, aggregator, and a server. As mentioned in Fig. 1, the previous layer provides model parameters of different clients through blockchain to global server. It passes this information into aggregator, that performs data aggregation and provides it to the global model. We used FedAvg, as an aggregator in our method that generate an average of parameter values after aggregation. Global model

update global parameters, and shared with participating clients. This process repeats for number of rounds mentioned in the experiment.

- **Initialization:** It presents majorly local training, aggregation, feature weighting, ROS computation, normalization, and ranking, consensus phase. It is first initialized with the ROS for device D_i , denoted by ROS_i . Initially, it is set to 0, and it indicates that the device is suitable for aggregation.
- **Local Model Training:** D_i, θ_{ij} represents model weights and biases learned by the ML model, $Acc_{ij}, F1_{ij}, AUC_{ij}$ represent an accuracy, f1-score, and are under the ROC curve, are used to evaluate the trained model's performance on a specific device.
- **Aggregation Phase:** Parametric M_j shows an ML model that relies on the fixed parameters. Non-parametric M_j is the ML model without a fixed number of parameters. θ_j^{global} denotes global model parameters obtained via fedavg aggregation. $\hat{y}^{global}$ is the global ensemble prediction, considered by the average local model predictions. Here, FedAvg takes a weighted step towards each client's local updated as per Eq. 9.

$$w_{t+1} = w_t + \sum_{k=1}^K \frac{n_k}{n} (w_t^k - w_t) \quad (9)$$

Here, w_t denotes the global model parameters at round t , w_t^k represents the locally updated model on client k , and $\frac{n_k}{n}$ is the weight assigned.

- **Feature Weighting Phase:** FI_k denotes feature importance value of feature k , derived from models. w_k represents the normalized feature weight.
- **ROS Computation Phase:** f_{ik} is the feature value with number k of device D_i . It represents resource metrics such as SNR, throughput, and energy. FC_i is the weighted sum that shows device performance across all features. p_i is the performance indicator of device D_i . The final resource optimization score is denoted by ROS_i , which combines p_i and FC_i .
- **Normalization and Ranking:** ROS'_i is calculated by ROS normalization into the [0,1] range for comparison across the devices. Rank D_i by ROS'_i indicates that the devices are arranged in descending order of normalized ROS to prioritize high-performing, resource-efficient nodes.
- **Consensus Phase:** As mentioned in Eq. 10, C_i is consensus value, calculated the how many times the device appears among the top-k ranked devices by various ML models. τ is the threshold for consensus, Select D_i with $C_i \geq \tau$ shows the devices are considered for further aggregation. Here,

$$C_i \leftarrow \frac{\text{Count}(D_i \in \text{Top-}K)}{\text{Total Models}} \quad (10)$$

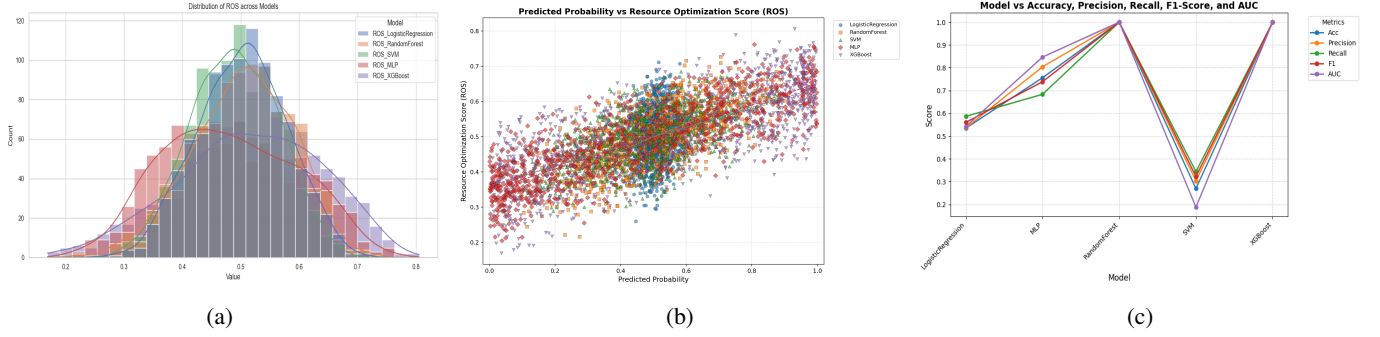


Fig. 2: (a) Model wise ROS, (b) Pred_Prob VS ROS for All Models, and (c) Global Model vs Evaluation Parameters.

- **Outputs:** ROS'_i is the final ROS value for all devices (100 in our case). Ranking is the sorted list of devices as per their normalized ROS. Selected devices are those devices that satisfy the threshold criteria.

III. RESULTS AND DISCUSSIONS

A. Federated learning evaluation

In this section, FL is evaluated in many phases, including ROS, predicted probability, ROS scattered, stacked contribution to ROS and global model performance. Based on this performance, we can conclude that our client provides resources to only optimal connections.

1) **Resource Optimization Score (ROS):** The ROS is calculated as mentioned in the previous section. Different model exhibits ROS spread as demonstrated in Fig. 2a. It shows that SVM and Logistic Regression report the highest ROS values.

2) **Predicted Probability:** It indicates probabilities of positive class, predicted by global model for each device. High value implies that the model is optimal, contributes in ROS increment despite of moderate contributions of other features. Various ML models in the federated set-up shows a predicted probability with respect to ROS as mentioned in Fig. 2b. Among all the models, MLP exhibits a very relevant pattern, as its prediction component is highly inclined towards ROS.

3) **Stacked Contributions to Resource Optimization Score (ROS):** This section clarifies how different features, contribute to the total ROS value for each device. Different features, such as social weight, channel gain, noise power, SNR, EDR, available bandwidth, inference level, resource block allocation, allocated throughput, fairness index, computational complexity, number of active connections, and probability contribution, create an impact on ROS calculation. Model-wise feature contribution for ROS calculation is mentioned in Fig. 3, shows that the prob_pred and throughput allocation create major impact on ROS calculation.

4) **Global Model Performance:** The overall global model performance is demonstrated in Fig.2c through different

evaluation parameters such as an accuracy, f1-score, AUC, recall, and precision value for 100 devices. It gives a detailed overview of how global models perform in FL settings. It is clearly noticeable that XGBoost and Random Forest are showing more than 95% in all the evaluation criteria.

B. Blockchain-based assessment

The blockchain performance is assessed using throughput and latency. To evaluate the performance of the proposed framework, we have integrated the Hyperledger Caliper. We configured HF with two organizations and 2 orderer node in a HF. We set the linear rate, HF SDK SUT 2.2, and the txnumber as parameters of the configuration file (Config.yaml).

Fig. 4a illustrates the throughput values which calculated by varying the number of transactions from 100 to 2000. This figure indicates that whenever the number of transactions increases, throughput often increases to some extent, leading to higher throughput values. The proposed framework confirms a larger number of transactions per second due to the smaller size of transactions, resulting in higher throughput than the conventional blockchain-based system.

Fig. 4b shows the latency values computed by varying the number of transactions. The proposed framework needs minimal data storage to store the CID of WB in the HFB. It reduces the time required to read the MP due to the small-scale size of the transactions. Hence, it leads to minimum latency, improving the network performance while reading the data.

Fig. 4c shows the number of transactions vs throughput to write model parameters in the blockchain. The proposed framework only stores the CID of WB and GWB in the blockchain. It lowers the processing overhead of transactions and achieves higher throughput values than a conventional blockchain-based system.

Fig. 4d depicts the number of transactions vs latency. The proposed framework saves only the CID instead of the entire file of WB . It results in a smaller transaction payload, reducing the propagation and transmission delay to write the data to the

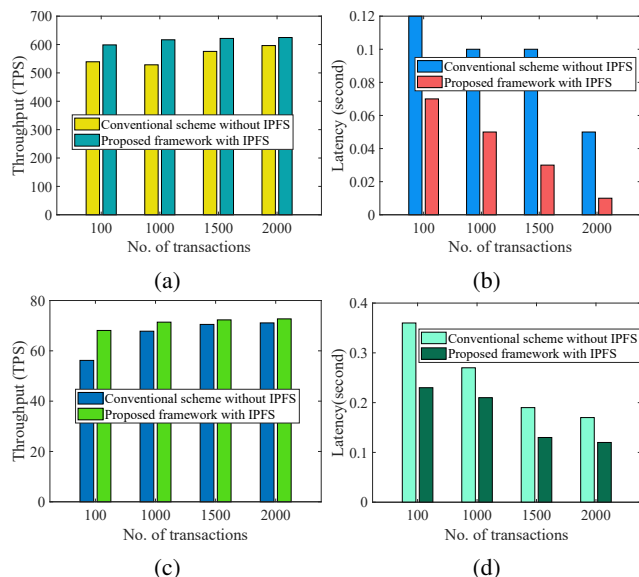
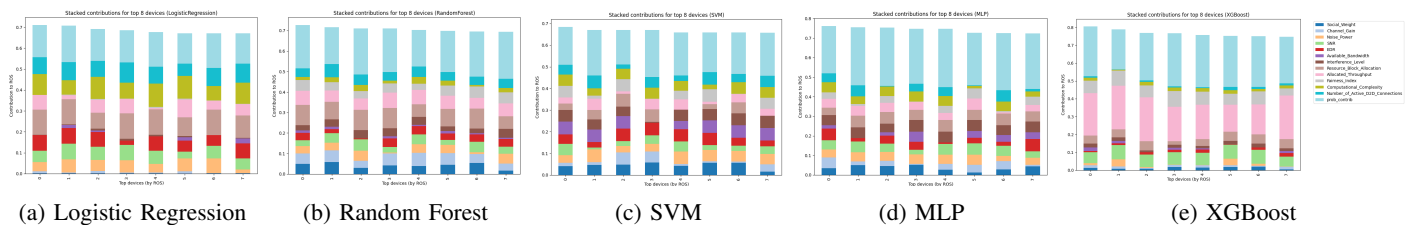


Fig. 4: No. of transactions in blockchain vs (a) Throughput for reading the model parameters, (b) Latency for reading the model parameters, (c) Throughput to write the model parameters, and (d) Latency to write the model parameters.

HFB. As a result, it achieves better latency than a conventional blockchain system.

IV. CONCLUSION AND FUTURE WORKS

In this paper, we presented FedChain, a secure and decentralized blockchain-enabled FL framework for resource allocation in D2D communication networks. By integrating HF with FL and utilizing IPFS for off-chain parameter storage, FedChain addresses the critical challenges of privacy leakage, single-point failures, and model tampering that arise in traditional centralized and standard FL-based systems. The use of a permissioned blockchain ensures transparent and immutable model update verification, while the off-chain storage mechanism maintains scalability with minimal communication overhead. Experimental evaluations indicate that FedChain achieves over 95% model accuracy with improved robustness, convergence stability, and security.

Overall, FedChain provides an effective foundation for secure collaborative intelligence in D2D environments and holds strong potential for deployment in future 6G edge intelligence ecosystems. In the future, we aim to extend FedChain to larger and more heterogeneous D2D networks and conduct experiments under dynamic network conditions that better reflect real-world scenarios. We plan to perform deployment-oriented evaluations using realistic network simulators to assess scalability, robustness, and latency under practical operating conditions. We also aim to integrate lightweight privacy-preserving mechanisms to further improve robustness and deployment efficiency.

REFERENCES

- [1] S. Shamaei, S. Bayat, and A. M. A. Hemmatyar, "Interference-aware resource allocation algorithm for d2d-enabled cellular networks using matching theory," *IEEE Transactions on Network and Service Management*, vol. 21, pp. 759–772, 2024.
- [2] M. Kamruzzaman, N. I. Sarkar, and J. A. Gutiérrez, "Machine learning-based resource allocation algorithm to mitigate interference in d2d-enabled cellular networks," *Future Internet*, vol. 16, p. 408, 2024.
- [3] R. Dangi and P. Lalwani, "Feature selection based machine learning models for 5g network slicing approximation," *Comput. Networks*, vol. 237, p. 110093, 2023.
- [4] H. M. F. Noman, K. Dimyati, K. A. Noordin, E. B. Hanafi, and A. Abdrabou, "Fedrl-d2d: Federated deep reinforcement learning-empowered resource allocation scheme for energy efficiency maximization in d2d-assisted 6g networks," *IEEE Access*, vol. 12, pp. 109775–109792, 2024.
- [5] Q. Guo, F. Tang, and N. Kato, "Federated reinforcement learning-based resource allocation in d2d-enabled 6g," *IEEE Network*, vol. 37, pp. 89–95, 2023.
- [6] J. Zhang, Y. Liu, X. Qin, X. Xu, and P. Zhang, "Adaptive resource allocation for blockchain-based federated learning in internet of things," *IEEE Internet of Things Journal*, vol. 10, pp. 10621–10635, 2023.
- [7] R. Cheng, Y. Sun, Y. Liu, L. Xia, D. Feng, and M. A. Imran, "Blockchain-empowered federated learning approach for an intelligent and reliable d2d caching scheme," *IEEE Internet of Things Journal*, vol. 9, pp. 7879–7890, 2021.
- [8] J. M. H. Pinheiro, S. V. B. de Oliveira, T. H. S. Silva, P. A. R. Saraiva, E. F. de Souza, L. A. Ambrosio, and M. Becker, "The impact of feature scaling in machine learning: Effects on regression and classification tasks," *ArXiv*, vol. abs/2506.08274, 2025.
- [9] J. R. Terven, D.-M. Córdova-Esparza, J.-A. Romero-González, A. Ramirez-Pedraza, and E. A. Chavez-Urbiola, "A comprehensive survey of loss functions and metrics in deep learning," *Artif. Intell. Rev.*, vol. 58, p. 195, 2023.